

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа

«Сложные алгоритмы поиска»

По дисциплине «Основы алгоритмизации и программирования»

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

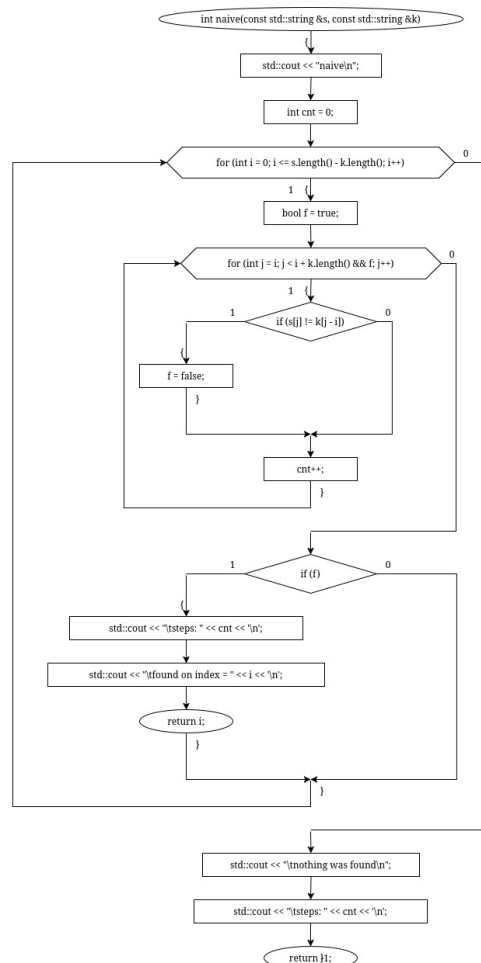
Реализовать алгоритмы поиска:

1. Наивный поиск подстроки в строке.
2. Поиск Боера-Мура.
3. Поиск Кнута-Морриса-Пратта.

АНАЛИЗ АЛГОРИТМА НАИВНОГО ПОИСКА

1. Последовательно перебираем каждый элемент строки циклом, затем ещё одним циклом попарно сравниваем элементы строки, начиная с текущего, с элементами образа. При несовпадении увеличиваем головной итератор на единицу. Иначе — нашли искомую подстроку.

БЛОК-СХЕМА



КОД

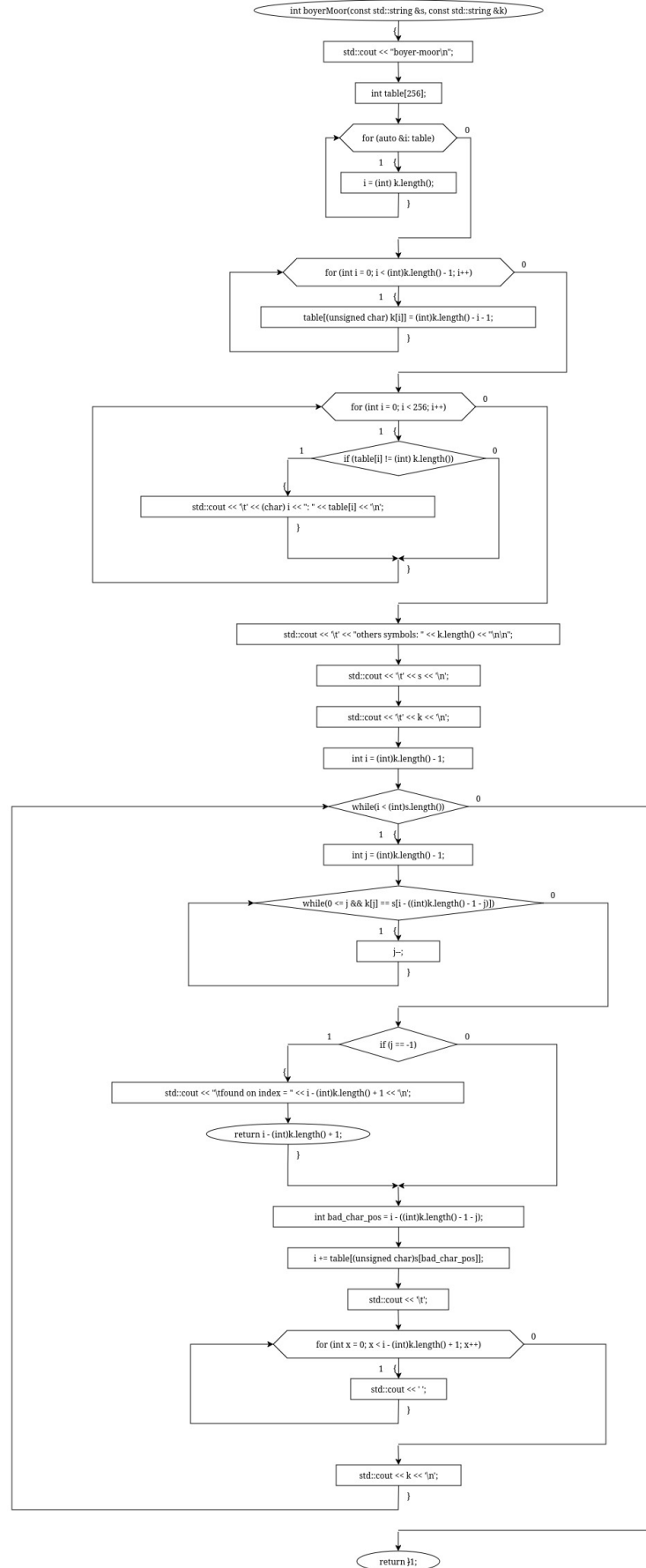
```
#include <string>
#include <iostream>

int naive(const std::string &s, const std::string &k) {
    std::cout << "naive\n";
    int cnt = 0;
    for (int i = 0; i <= s.length() - k.length(); i++) {
        bool f = true;
        for (int j = i; j < i + k.length() && f; j++) {
            if (s[j] != k[j - i]) f = false;
            cnt++;
        }
        if (f) {
            std::cout << "\tsteps: " << cnt << '\n';
            std::cout << "\tfound on index = " << i << '\n';
            return i;
        }
    }
    std::cout << "\tnothing was found\n";
    std::cout << "\tsteps: " << cnt << '\n';
    return -1;
}
```

АНАЛИЗ АЛГОРИТМА БОЕРА-МУРА

1. Создаётся таблица смещений table[256]: для каждого символа записывается расстояние от его последнего появления в шаблоне до конца строки k. Для отсутствующих символов и последнего сдвиг равен длине шаблона.
2. Сравнение шаблона с текстом выполняется справа налево, начиная с последнего символа k. При несовпадении шаблон сдвигается на значение table[s[позиция_несовпавшего_в_исходной_строке_символа]]. Таким образом пропускаются заведомо неподходящие позиции и уменьшает число сравнений.

БЛОК-СХЕМА



КОД

```
#include <string>
#include <iostream>

int boyerMoor(const std::string &s, const std::string &k) {
    std::cout << "boyer-moor\n";
    int table[256];
    for (auto &i: table) i = (int) k.length();
    for (int i = 0; i < (int)k.length() - 1; i++)
        table[(unsigned char) k[i]] = (int)k.length() - i - 1;

    for (int i = 0; i < 256; i++)
        if (table[i] != (int) k.length())
            std::cout << '\t' << (char) i << ": " << table[i] << '\n';
    std::cout << '\t' << "others symbols: " << k.length() << "\n\n";
    std::cout << '\t' << s << '\n';
    std::cout << '\t' << k << '\n';

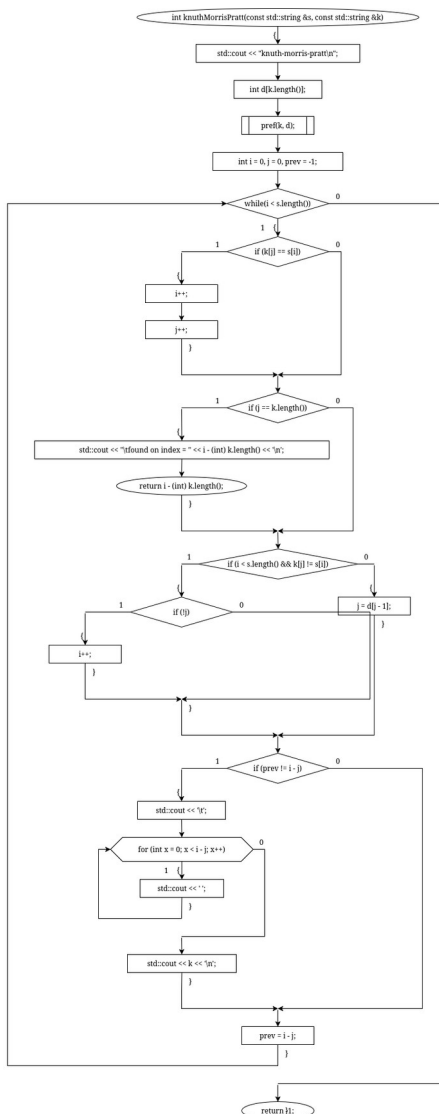
    int i = (int)k.length() - 1;
    while (i < (int)s.length()) {
        int j = (int)k.length() - 1;
        while (0 <= j && k[j] == s[i - ((int)k.length() - 1 - j)])
            j--;
        if (j == -1) {
            std::cout << "\tfound on index = " << i - (int)k.length() + 1 << '\n';
            return i - (int)k.length() + 1;
        }
        int bad_char_pos = i - ((int)k.length() - 1 - j);
        i += table[(unsigned char)s[bad_char_pos]];

        std::cout << '\t';
        for (int x = 0; x < i - (int)k.length() + 1; x++) std::cout << ' ';
        std::cout << k << '\n';
    }
    return -1;
}
```

АНАЛИЗ АЛГОРИТМА КНУТА-МОРРИСА-ПРАТТА

1. Префикс строки — начало строки, суффикс — её конец. Префикс-функция $d[i]$ хранит длину наибольшего собственного префикса подстроки $s[0..i]$, совпадающего с её суффиксом.
- 2 Для шаблона k предварительно вычисляется массив d , позволяющий определить, на сколько символов можно сдвинуть шаблон при несовпадении.
3. Сравнение текста и шаблона выполняется слева направо с помощью индексов i и j .
4. При несовпадении символов индекс j изменяется на $d[j - 1]$, то есть поиск продолжается с позиции предыдущего возможного совпадающего префикса без повторной проверки символов текста.

БЛОК-СХЕМА



КОД

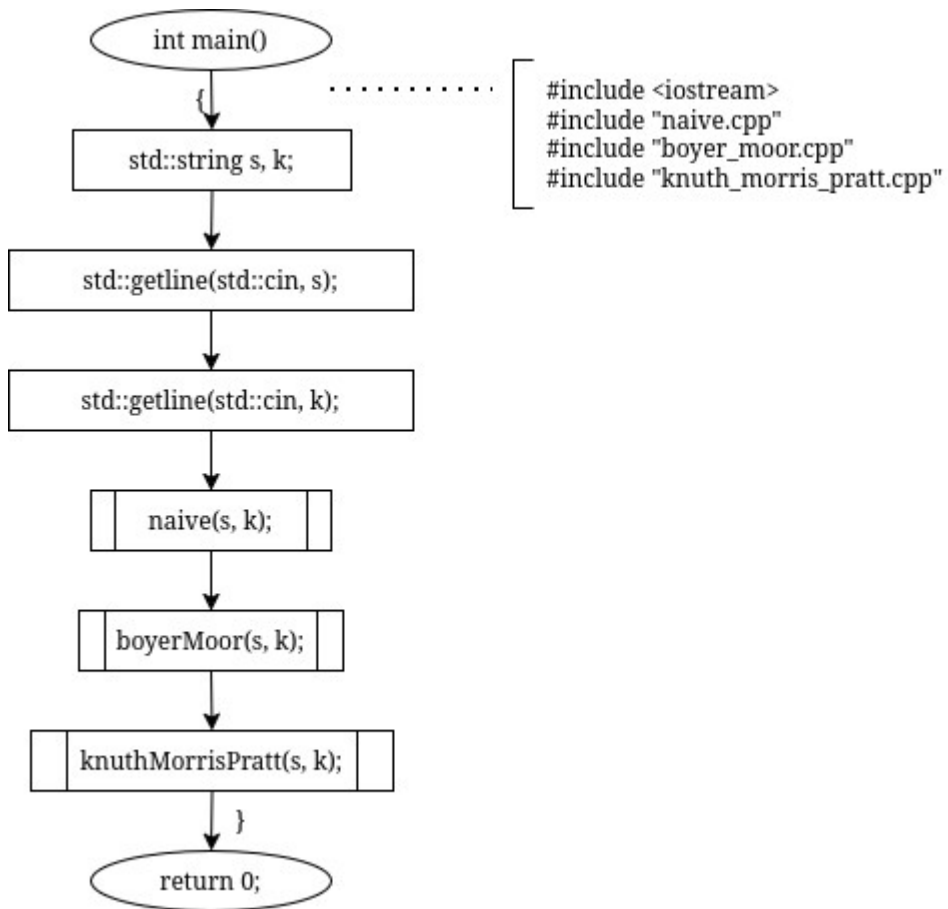
```
#include <string>
#include <iostream>
void pref(const std::string &s, int *d) {
    d[0] = 0;
    int j = 0, i = 1;
    while (i < s.length()) {
        if (s[i] == s[j]) d[i++] = ++j;
        else if (!j) d[i++] = 0;
        else j = d[j - 1];
    }
    for (int i = 0; i < s.length(); i++) {
        std::cout << '\t' << (char) s[i] << ": " << d[i] << '\n';
    }
}

int knuthMorrisPratt(const std::string &s, const std::string &k) {
    std::cout << "knuth-morris-pratt\n";
    int d[k.length()];
    pref(k, d);

    int i = 0, j = 0, prev = -1;
    while (i < s.length()) {
        if (k[j] == s[i]) {
            i++;
            j++;
        }
        if (j == k.length()) {
            std::cout << "\tfound on index = " << i - (int) k.length() << '\n';
            return i - (int) k.length();
        }
        if (i < s.length() && k[j] != s[i])
            if (!j) i++;
            else j = d[j - 1];

        if (prev != i - j) {
            std::cout << '\t';
            for (int x = 0; x < i - j; x++) std::cout << ' ';
            std::cout << k << '\n';
        }
        prev = i - j;
    }
    return -1;
}
```

БЛОК-СХЕМА ФУНКЦИИ int main()



КОД

```
#include <iostream>
#include "naive.cpp"
#include "boyer_moor.cpp"
#include "knuth_morris_pratt.cpp"
```

```
int main() {
    std::string s, k;
    std::getline(std::cin, s);
    std::getline(std::cin, k);

    naive(s, k);
    boyerMoor(s, k);
    knuthMorrisPratt(s, k);
}
```


РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ

ABC ABCDAB ABCDABCDABDE

ABCDABD

naive

steps: 39

found on index = 15

boyer-moor

A: 2

B: 1

C: 4

D: 3

others symbols: 7

ABC ABCDAB ABCDABCDABDE

ABCDABD

ABCDABD

ABCDABD

ABCDABD

found on index = 15

knuth-morris-pratt

A: 0

B: 0

C: 0

D: 0

A: 1

B: 2

D: 0

ABCDABD

ABCDABD

ABCDABD

ABCDABD

ABCDABD

ABCDABD

ABCDABD

found on index = 15